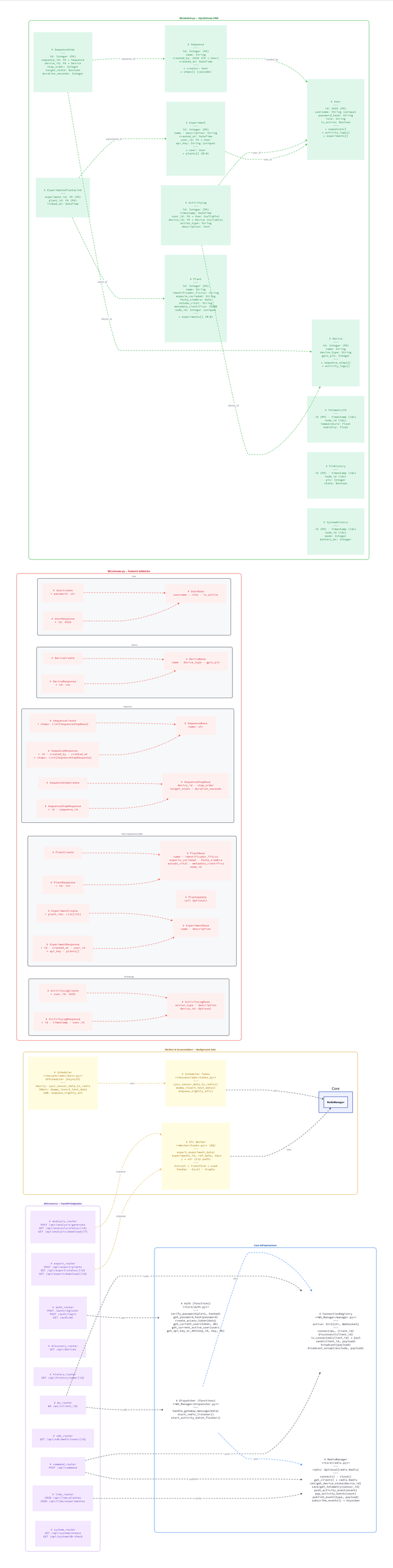

BD/models.py — SQLAlchemy ORM

```
# SequenceStep
---
id: Integer (PK)
sequence_id: FK → Sequence
device_id: FK → Device
step_order: Integer
target_state: Boolean
duration_seconds: Integer
```

```
# Sequence
---
id: Integer (PK)
name: String
created_by: UUID (FK → User)
created_at: DateTime
---
→ creator: User
→ steps[] (cascade)
```

```
# User
---
id: UUID (PK)
username: String (unique)
password_hash: String
role: String
is_active: Boolean
---
→ sequences[]
→ activity_logs[]
→ experiments[]
```

```
# Experiment
---
id: Integer (PK)
name · description: String
created_at: DateTime
user_id: FK → User
api_key: String (unique)
---
→ user: User
→ plants[] (M:N)
```

```
# ExperimentoPlantaLink
---
experiment_id: FK (PK)
plant_id: FK (PK)
linked_at: DateTime
```

```
# ActivityLog
---
id: Integer (PK)
timestamp: DateTime
user_id: FK → User (nullable)
device_id: FK → Device (nullable)
action_type: String
description: Text
```

```
# Plant
---
id: Integer (PK)
name: String
identificador_fisico: String
especie_variedad: String
fecha_siembra: Date
estado_vital: String
metadata_cientifica: JSONB
node_id: Integer (unique)
---
→ experiments[] (M:N)
```

```
# Device
---
id: Integer (PK)
name: String
device_type: String
gpio_pin: Integer
---
→ sequence_steps[]
→ activity_logs[]
```

```
# TelemetryTH
---
id (PK) · timestamp (idx)
node_id (idx)
temperature: Float
humidity: Float
```

```
# PinHistory
---
id (PK) · timestamp (idx)
node_id (idx)
pin: Integer
state: Boolean
```

```
# SystemHistory
---
id (PK) · timestamp (idx)
node_id (idx)
mode: Integer
battery_mv: Integer
```

BD/schemas.py — Pydantic Validation

User

```
# UserCreate
+ password: str
```

```
# UserBase
username · role · is_active
```

```
# UserResponse
+ id: UUID
```

Device

```
# DeviceCreate
```

```
# DeviceBase
name · device_type · gpio_pin
```

```
# DeviceResponse
+ id: int
```

Sequence

```
# SequenceCreate
+ steps: List[SequenceStepBase]
```

```
# SequenceBase
name: str
```

```
# SequenceResponse
+ id · created_by · created_at
+ steps: List[SequenceStepResponse]
```

```
# SequenceStepCreate
```

```
# SequenceStepBase
device_id · step_order
target_state · duration_seconds
```

```
# SequenceStepResponse
+ id · sequence_id
```

Plant / Experiment (LIMS)

```
# PlantCreate
```

```
# PlantBase
name · identificador_fisico
especie_variedad · fecha_siembra
estado_vital · metadata_cientifica
node_id
```

```
# PlantResponse
+ id: int
```

```
# PlantUpdate
(all Optional)
```

```
# ExperimentCreate
+ plant_ids: List[int]
```

```
# ExperimentBase
name · description
```

```
# ExperimentResponse
+ id · created_at · user_id
+ api_key · plants[]
```

ActivityLog

```
# ActivityLogCreate
+ user_id: UUID
```

```
# ActivityLogBase
action_type · description
device_id: Optional
```

```
# ActivityLogResponse
+ id · timestamp · user_id
```

Worker/ & Secuenciador/ — Background Jobs

```
# Scheduler
**Secuenciador/main.py**
APScheduler (AsyncIO)
---
Hourly: sync_sensor_data_to_redis
30min: dummy_insert_test_data
3AM: enqueue_nightly_etl
```

```
# Scheduler Tasks
**Secuenciador/tasks.py**
---
sync_sensor_data_to_redis()
dummy_insert_test_data()
enqueue_nightly_etl()
```

```
# ETL Worker
**Worker/tasks.py** (RQ)
---
export_experiment_data(
experimento_id, ref_date, days
) → str (zip path)
---
Extract → Transform → Load
Pandas · Excel · Graphs
```

API/routers/ — FastAPI Endpoints

```
# analysis_router
POST /api/analysis/generate
GET /api/analysis/status/{id}
GET /api/analysis/download/{f}
```

```
# export_router
POST /api/export/plants
GET /api/export/status/{id}
GET /api/export/download/{id}
```

```
# auth_router
POST /auth/register
POST /auth/login
GET /auth/me
```

```
# discovery_router
GET /api/devices
```

```
# history_router
GET /api/history/node/{id}
```

```
# ws_router
WS /ws/{client_id}
```

```
# sdk_router
GET /api/sdk/mediciones/{id}
```

```
# command_router
POST /api/command
```

```
# lims_router
CRUD /api/lims/plantas
CRUD /api/lims/experimentos
```

```
# system_router
GET /api/system/status
GET /api/system/db-check
```

Core Infrastructure

```
# Auth (functions)
**Core/auth.py**
---
verify_password(plain, hashed)
get_password_hash(password)
create_access_token(data)
get_current_user(token, db)
get_current_active_user(user)
get_api_key_or_403(exp_id, key, db)
```

```
# ConnectionRegistry
**WS_Manager/manager.py**
---
active: Dict[str, WebSocket]
---
connect(ws, client_id)
disconnect(client_id)
is_connected(client_id) → bool
send(client_id, payload)
broadcast(payload)
broadcast_except(exclude, payload)
```

```
# Dispatcher (functions)
**WS_Manager/dispatcher.py**
---
handle_gateway_message(data)
start_redis_listener()
start_activity_batch_flusher()
```

```
# RedisManager
**Core/redis.py**
---
redis: Optional[redis.Redis]
---
connect() · close()
get_client() → redis.Redis
set/get_device_state(device_id)
save/get_telemetry(sensor_id)
push_activity_event(event)
pop_activity_batch(count)
publish_event(type, payload)
subscribe_events() → AsyncGen
```